

BUILDER DESIGN PATTERN

1. THE PROBLEM

When creating complex objects like an `HttpRequest`, using standard constructors leads to "constructor telescoping"—a massive, unmanageable list of overloaded constructors for every possible combination of fields. If you try to fix this by using a default constructor and setter methods, you introduce three new problems: the object becomes mutable, it can enter an inconsistent state if the client forgets a mandatory setter, and you are forced to scatter if-else validation checks throughout your entire codebase before executing the object.

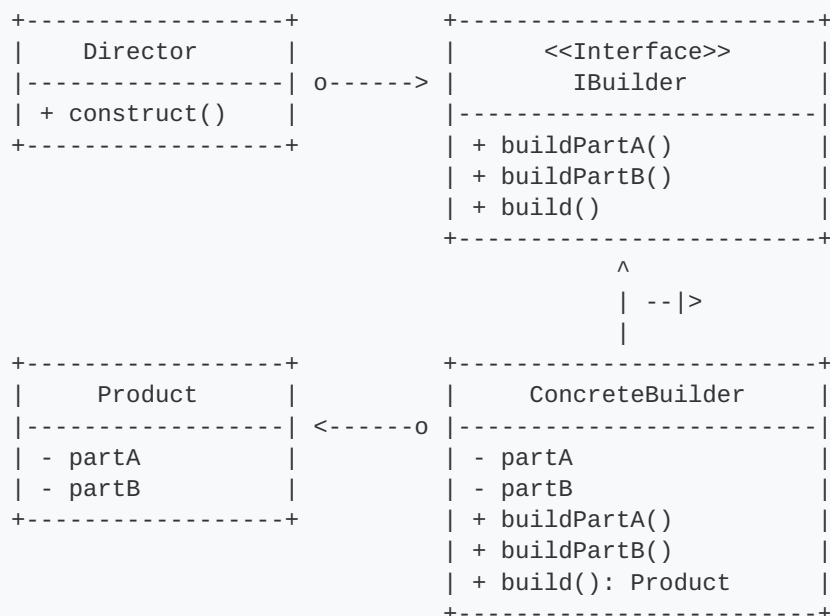
2. Intuition: Think of ordering a custom pizza or a Subway sandwich. You don't just ask for a "sandwich" and receive a random one, nor do you try to hand the sandwich artist every single ingredient at the exact same time (a giant constructor). Instead, you use a step-by-step process: you choose the bread, then the meat, then the veggies. Only when you are completely finished specifying your choices do they assemble the final, complete sandwich and hand it to you. Once handed to you, the sandwich's core build is complete and immutable.

3. KEY COMPONENTS

COMPONENT	ROLE	EXAMPLE
Product	The complex object being constructed, completely immutable.	<code>HttpRequest</code>
Builder	The interface/class that provides chained methods to configure the product.	<code>HttpRequestBuilder</code>

Director	(Optional) Contains predefined templates to build common configurations automatically.	ApiRequestDirector
Step Interfaces	(Optional) Forces a strict sequence of method calls at compile-time.	UrlStep, MethodStep

4. UML CLASS DIAGRAM



5. HOW IT WORKS — STEP BY STEP

1. Client wants to create an object but cannot use the `new` keyword directly because the `Product`'s constructor is private.
2. Client instantiates the `Builder` object (or calls a static `Builder::create()` method).

3. Client calls a series of intermediate methods (like `withUrl()`, `withMethod()`) via Method Chaining. Each method modifies the Builder's internal state and returns a reference to the Builder itself (`return *this`).
4. Client calls the terminal `build()` method on the Builder.
5. Inside `build()`, the Builder runs centralized validation checks. If a mandatory field is missing, it throws an error.
6. **Result:** If validation passes, `build()` calls the private constructor of the Product, passing in the configured values, and returns a fully constructed, immutable Product object to the Client.

6. CODE SKELETON (C++)

```
#include <iostream>
#include <string>
#include <string>
#include <string>

class HttpRequest; // Forward declaration

// BUILDER CLASS
class HttpRequestBuilder {
private:
    std::string url;
    std::string method;
    std::string body;

public:
    HttpRequestBuilder() : method("GET") {} // Default value

    // Intermediate methods (Method Chaining)
    HttpRequestBuilder& withUrl(std::string u) {
        this->url = u;
        return *this;
    }

    HttpRequestBuilder& withMethod(std::string m) {
        this->method = m;
        return *this;
    }
}
```

```

        HttpRequestBuilder& withBody(std::string b) {
            this->body = b;
            return *this;
        }

        // Terminal method with centralized validation
        HttpRequest build();
    };

    // PRODUCT CLASS
    class HttpRequest {
    private:
        std::string url;
        std::string method;
        std::string body;

        // Private constructor - Client cannot call this directly!
        HttpRequest(std::string u, std::string m, std::string b)
            : url(u), method(m), body(b) {}

        // Grant Builder access to the private constructor
        friend class HttpRequestBuilder;

    public:
        void execute() {
            std::cout << "Executing " << method << " request to " << url <<
            "\n";
        }
    };

    // Implementing the build method after HttpRequest is fully defined
    HttpRequest HttpRequestBuilder::build() {
        if (url.empty()) {
            throw std::invalid_argument("URL is mandatory!");
        }
        if (method == "POST" && body.empty()) {
            throw std::invalid_argument("POST requests must have a body!");
        }
        return HttpRequest(url, method, body); // Safe, immutable creation
    }

    // DIRECTOR CLASS (Enhancement for reusable templates)
    class ApiRequestDirector {
    public:
        static HttpRequest createStandardGetRequest(std::string url) {
            return HttpRequestBuilder()
                .withUrl(url)
                .withMethod("GET")
                .build();
        }
    };

```

```
};

// CLIENT USAGE
int main() {
    // Standard Builder usage via Method Chaining
    HttpRequestBuilder builder;
    HttpRequest req1 = builder.withUrl("https://api.example.com")
                              .withMethod("POST")
                              .withBody("{\"id\": 1}")
                              .build();

    req1.execute();

    // Using the Director for a predefined template
    HttpRequest req2 =
    ApiRequestDirector::createStandardGetRequest("https://google.com");
    req2.execute();

    return 0;
}
```

7. REAL-WORLD USE CASES

- **HTTP Request/Response Libraries:** Building network requests with combinations of headers, bodies, timeouts, and auth tokens cleanly (as seen above).
- **SQL Query Builders:** Constructing complex database queries dynamically (e.g., SELECT -> FROM -> WHERE) using method chaining.
- **Test Data Generation:** Creating mock objects with very specific, complex states for unit testing without writing dozens of messy constructors.

8. INTERVIEW QUICK REFERENCE

Definition: Builder separates the construction of a complex object from its representation, ensuring objects are built safely, validated centrally, and kept immutable.

When to use:

- When dealing with constructor telescoping (too many constructor parameters).

- When you want to enforce object immutability but have complex initialization.
- When an object requires step-by-step configuration and strict validation before it can be safely used.

Commonly confused with: *Factory Pattern*. A Factory creates an object in a single step and hides the creation logic completely. A Builder creates an object step-by-step and requires the client to explicitly configure the details before finalizing.

Key interview questions:

Q1: How does the Step Builder enhancement work, and what problem does it solve?

A1: It solves the problem of clients calling builder methods out of order or skipping mandatory steps. It uses distinct pure abstract classes/interfaces for each step. When a client calls `withUrl()`, the method doesn't return the generic builder; it returns a `MethodStep` interface, forcing the compiler/IDE to restrict the client's next action to `withMethod()` only.

Q2: Why is the target class constructor made private, and how does the builder access it?

A2: It is made private to prevent clients from bypassing the builder and creating objects in an inconsistent state. The builder accesses it by being declared as an inner static class (in Java) or a friend class (in C++).

Q3: What is the purpose of the Director class in the Builder pattern?

A3: The Director encapsulates predefined, frequently used configurations (like a standard JSON POST request template) so the client doesn't have to manually write out the same long method chain every time they want a common object.